Parallel DAG-Based Task Scheduling with Dynamic Load Balancing using Work-Stealing

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Mannya Soni	102316067
2	Aditya Chauhan	102496001
3	Gautam Tayal	102483024

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2025

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

2 Literature Review **2**

2.1 Research Gaps 4

3 Problem Formulation **5**

3.1 Task Dependency Constraint 5

3.2 Parallel Execution Model 6

3.3 Objective Function 6

3.4 Optimization Goals 6

3.5 Work-Stealing Strategy 6

4 Objectives **7**

5 Methodology **7**

5.1 DAG Construction 7

5.2 Task Initialization 7

5.3 Thread Pool Design 8

5.4 Task Execution Process 8

5.5 Work-Stealing Mechanism 8

5.6 Scheduling Algorithm 8

5.7 Synchronization and Concurrency Control 9

5.8 Performance Evaluation 9

6 Dataset Description **10**

6.1 DAG Generation 10

6.2 Task Characteristics 10

6.3 Dataset Size and Configuration 10

6.4 Purpose of Dataset 11

7 Results and Discussion **11**

7.1 Performance Results 11

7.2 Performance Visualization 11

7.3 Discussion 12

8 Conclusion **13**

List of Figures

1 Execution Time and Speedup vs Thread Count 12

List of Tables

1 Summary of Related Work in DAG Scheduling and Work-Stealing 3
2 Performance Evaluation of Parallel Scheduler 11

1 Introduction

1.1 Background

The rapid increase in the number of processor cores over the last two decades has led to parallel programming being an essential part of software engineering and HPC. Todayâs computers often come equipped with multiple processor cores within a single microchip, while systems with shared memory and hundreds of hardware threads are becoming common. [1]. An efficient application of hardware parallelism requires adequate workload distribution and task scheduling.

In many real-life situations, such as computational science, multiple tasks are present; however, there are interdependencies between them. It is clear that these computations cannot be executed in an embarrassingly parallel way because of the existence of interdependencies between tasks. It becomes obvious that the description of these interdependencies can be performed using the concept of Directed Acyclic Graph (DAG), where tasks correspond to nodes, and the edges represent the dependencies between the tasks. One needs to take into account all these dependencies to guarantee the accuracy of the computations.

Generally, static scheduling approaches have been used in allocating tasks to processors. These approaches are straightforward and simple to implement, but it is impossible to offer any performance guarantees in the presence of imbalanced workloads due to different task execution times. Therefore, static algorithms produce poor load balancing when some processors are overloaded and others are underutilized. To overcome these issues, the concept of dynamic scheduling was proposed.[2].

1.2 Problem Statement

Consider a computational workflow consisting of n tasks $\{t_0, t_1, \ldots, t_{n-1}\}$, where each task may depend on one or more preceding tasks. This workflow can be represented as a **Directed Acyclic Graph (DAG)** $G = (V, E)$, where V denotes the set of tasks and a directed edge $(u, v) \in E$ indicates that task u must complete before task v can begin. The acyclic property ensures that a valid execution order exists.

The scheduling problem involves assigning tasks in DAG G to T processors such that:

1. All task dependency constraints are strictly satisfied to ensure correct execution order.
2. The overall completion time (makespan) is minimized to achieve efficient parallel performance.
3. Processor idle time is reduced to maximize resource utilization and improve system efficiency.

This scheduling problem is known to be NP-hard in the general case [3], making optimal solutions computationally infeasible for large-scale task graphs. As a result, practical systems rely on heuristic and approximation techniques to achieve near-optimal performance.

One of the most effective dynamic scheduling strategies is work-stealing, in which idle threads dynamically acquire tasks from the queues of busy threads. This approach enables decentralized load balancing and has been shown to achieve efficient execution with strong theoretical guarantees for a wide range of DAG-based computations [1].

To address the challenges of dependency-aware scheduling and load imbalance, this project proposes a parallel DAG-based task scheduling system implemented in modern C++17. The system employs a fully dynamic and decentralized work-stealing mechanism to efficiently utilize processing resources, minimize execution time, and improve scalability in shared-memory multi-core environments.

2 Literature Review

The scheduling of task DAGs has been studied extensively in both theoretical computer science and systems research.

Table 1: Summary of Related Work in DAG Scheduling and Work-Stealing

Author(s) & Year	**Approach**	**Key Contribution**	**Limitation**
Blumofe & Leiserson (1999) [1]	Work-Stealing Scheduling for Multithreaded Computations	Introduced the work-stealing algorithm with theoretical guarantees for efficient parallel execution and near-optimal makespan	Assumes ideal conditions; does not fully address practical overheads like synchronization costs
Kwok & Ahmad (1999) [3]	Static DAG Scheduling Algorithms	Provided a comprehensive survey of static scheduling techniques for DAGs in multiprocessor systems	Static scheduling lacks adaptability to runtime workload variations and dynamic environments
Arora et al. (2001) [4]	Thread Scheduling for Multiprocessors	Proposed improved theoretical models for scheduling multithreaded computations with better load balancing	Focuses more on theoretical guarantees than practical system implementation
Robison (2008) [2]	Intel Threading Building Blocks (TBB)	Demonstrated practical implementation of dynamic scheduling and task parallelism using work-stealing	Requires external libraries and abstraction layers, reducing low-level control over scheduling

The literature review presented in Table 1 highlights the evolution of task scheduling techniques for DAG-based workloads in parallel computing systems. Early research by Blumofe and Leiserson [1] established work-stealing as an efficient dynamic scheduling

strategy, providing strong theoretical guarantees for minimizing execution time in multithreaded environments. In contrast, the survey by Kwok and Ahmad [3] focused on static scheduling algorithms, which, although simple and predictable, lack adaptability to runtime variations and often result in load imbalance. Further theoretical advancements by Arora et al. [4] improved scheduling models for multiprocessor systems, emphasizing better load distribution but with limited focus on practical implementation challenges. Robison [2] demonstrated the real-world applicability of dynamic scheduling through Intelâs Threading Building Blocks (TBB), which employs work-stealing for efficient task parallelism, though it relies on external abstractions.

In general, one may conclude that while the static scheduling scheme fails to accommodate fluctuating loads, a dynamic scheduling technique such as work stealing emerges as the optimal solution. The issue with all existing implementations is that they either overly emphasize theoretical considerations or are heavily dependent on some other software packages.

2.1 Research Gaps

Based on the review of existing literature, the following research gaps have been identified:

- **Lack of Adaptability in Static Scheduling:** Static scheduling approaches [3] assume prior knowledge of task execution characteristics and fail to adapt to runtime variations, leading to poor load balancing in irregular workloads.

- **Gap Between Theory and Practical Implementation:** While work-stealing techniques [1] provide strong theoretical guarantees, many studies focus on analytical models rather than practical system-level implementations, limiting real-world applicability.

- **Dependence on External Frameworks:** Existing implementations such as Intel Threading Building Blocks (TBB) [2] rely on high-level libraries and abstractions, reducing transparency and limiting fine-grained control over scheduling behavior.

- **Limited Focus on Dynamic Dependency Management:** Many scheduling systems do not fully address real-time updates of task dependencies in DAG-based

workflows, which is essential for ensuring correctness in dynamic execution environments.

- **Lack of Lightweight and Self-Contained Systems:** There is a shortage of simple, standalone scheduling frameworks that integrate DAG-based dependency handling with dynamic load balancing using only standard programming constructs.

To address these gaps, the proposed system implements a fully dynamic, decentralized DAG scheduling framework using work-stealing in standard C++17, without relying on external libraries. The system focuses on both correctness and performance, bridging the gap between theoretical models and practical implementation.

3 Problem Formulation

The task scheduling problem in this project is modeled using a Directed Acyclic Graph (DAG), defined as:

$$G = (V, E) \tag{1}$$

where:

- $V = \{t_0, t_1, \ldots, t_{n-1}\}$ represents the set of n tasks
- $E \subseteq V \times V$ represents the set of directed edges (dependencies)

A directed edge $(u, v) \in E$ indicates that task u must be completed before task v can begin execution. The graph is acyclic, ensuring that a valid execution order exists.

3.1 Task Dependency Constraint

Each task t_i is associated with a dependency count:

$$dep(t_i) = \text{number of unresolved dependencies of } t_i \tag{2}$$

A task is considered *ready* for execution if:

$$dep(t_i) = 0 \tag{3}$$

3.2 Parallel Execution Model

Let $T = \{P_1, P_2, \ldots, P_m\}$ denote the set of m processors (threads). The objective is to assign tasks to processors such that multiple independent tasks can be executed concurrently while respecting dependency constraints.

3.3 Objective Function

The primary objective is to minimize the total execution time (makespan), defined as:

$$C_{\max} = \max_{t_i \in V} \{C(t_i)\} \tag{4}$$

where $C(t_i)$ represents the completion time of task t_i.

3.4 Optimization Goals

The scheduling problem aims to satisfy the following goals:

- **Dependency Satisfaction:** All precedence constraints must be maintained.
- **Minimization of Makespan:** Total completion time should be minimized.
- **Load Balancing:** Tasks should be evenly distributed across processors.
- **Maximized Resource Utilization:** Idle time of processors should be minimized.

3.5 Work-Stealing Strategy

To address load imbalance, a dynamic work-stealing strategy is employed. Each processor maintains a local task queue. When a processor becomes idle, it attempts to steal tasks from the queue of another processor. This dynamic redistribution helps in maintaining load balance and improving overall system performance.

Thus, the problem reduces to designing an efficient scheduling strategy that dynamically assigns ready tasks to processors while minimizing makespan and ensuring balanced workload distribution.

4 Objectives

The objectives of this project are as follows:

- **Model Tasks Using DAG:** To represent computational workflows as a Directed Acyclic Graph (DAG) with clearly defined task dependencies.
- **Enable Parallel Execution:** To implement a multi-threaded system that executes independent tasks concurrently using C++ concurrency features.
- **Ensure Dependency-Aware Scheduling:** To execute tasks only when all their dependencies are satisfied, ensuring correctness.
- **Implement Work-Stealing for Load Balancing:** To dynamically balance workload by allowing idle threads to steal tasks from busy threads.
- **Evaluate Performance and Scalability:** To analyze execution time, speedup, and efficiency, and assess system scalability with increasing threads.

5 Methodology

The proposed system implements a parallel DAG-based task scheduling framework using multi-threading and dynamic load balancing through work-stealing. The methodology consists of the following key components:

5.1 DAG Construction

The computational workflow is modeled as a Directed Acyclic Graph (DAG), where each node represents a task and directed edges represent dependencies between tasks. Each task maintains a dependency counter indicating the number of unresolved dependencies. Tasks with zero dependency count are considered ready for execution.

5.2 Task Initialization

Initially, all tasks are created and their dependencies are established based on the DAG structure. Tasks with no incoming edges (i.e., dependency count = 0) are identified and

inserted into the ready queues for execution.

5.3 Thread Pool Design

A fixed-size thread pool is created using C++ concurrency libraries (`std::thread`, `mutex`, `atomic`). Each worker thread is associated with a local double-ended queue (deque) that stores tasks assigned to it.

5.4 Task Execution Process

Each worker thread continuously performs the following steps:

1. Fetch a task from its local queue.
2. Execute the task.
3. Upon completion, update the dependency counters of its dependent tasks.
4. If any dependent task's counter becomes zero, it is added to a queue for execution.

5.5 Work-Stealing Mechanism

To ensure load balancing, a work-stealing strategy is implemented. If a thread's local queue becomes empty, it attempts to steal tasks from the front of another threadâs queue. This mechanism helps redistribute workload dynamically and prevents threads from remaining idle.

5.6 Scheduling Algorithm

The overall task scheduling process using work-stealing is described in Algorithm 1.

Algorithm 1 Parallel DAG Scheduling using Work-Stealing

```
Initialize DAG G = (V, E)
Compute dependency count for each task
Insert all tasks with dep(t_i) = 0 into ready queues
while there exist unfinished tasks do
    for all worker threads T_j in parallel do
        task ← pop from local queue
        if task = NULL then
            task ← steal from another thread's queue
        end if
        if task ≠ NULL then
            Execute task
            for all dependent tasks t_k of task do
                Decrement dep(t_k) atomically
                if dep(t_k) = 0 then
                    Insert t_k into a ready queue
                end if
            end for
        end if
    end for
end while
```

5.7 Synchronization and Concurrency Control

To ensure thread safety, synchronization primitives such as mutexes and atomic variables are used. Dependency counters are updated atomically to avoid race conditions, and access to task queues is controlled using locks where necessary.

5.8 Performance Evaluation

The system is evaluated by comparing sequential and parallel execution. Experiments are conducted with varying numbers of threads, and performance metrics such as execution time, speedup, and efficiency are measured to assess scalability and effectiveness of the

proposed approach.

6 Dataset Description

In this project, no external dataset is used. Instead, a synthetic workload is generated in the form of Directed Acyclic Graphs (DAGs) to simulate real-world task dependencies.

6.1 DAG Generation

The task graph is programmatically generated using a structured approach to mimic realistic dependency patterns. In this implementation, a tree-based DAG structure is used, where each task may generate multiple dependent child tasks. This results in a hierarchical graph with multiple levels of parallelism.

6.2 Task Characteristics

Each node in the DAG represents an independent computational task. The following characteristics define the workload:

- Each task has a unique identifier.
- Tasks maintain a dependency counter representing the number of prerequisite tasks.
- Tasks simulate computation using a fixed workload (e.g., iterative loop) to represent execution time.

6.3 Dataset Size and Configuration

To evaluate system performance, multiple DAGs of varying sizes are generated. The experiments are conducted with different configurations, including:

- Number of tasks (e.g., small, medium, large DAGs)
- Depth of the DAG (levels of dependencies)
- Number of threads (e.g., 1, 2, 4, 8, 16)

This variation allows analysis of scalability and performance under different workload conditions.

6.4 Purpose of Dataset

The synthetic DAG dataset is designed to:

- Simulate dependency-driven task execution
- Evaluate parallel scheduling efficiency
- Analyze load balancing behavior under dynamic conditions
- Measure performance improvements over sequential execution

The use of synthetic data provides full control over task structure and complexity, enabling consistent and repeatable performance evaluation.

7 Results and Discussion

7.1 Performance Results

As shown in Table 2, the execution time decreases significantly with an increase in the number of threads, indicating effective parallel execution. The system achieves a speedup of up to 7.05x with 16 threads. Although efficiency decreases at higher thread counts due to synchronization and overheads, overall performance improvement remains substantial.

Table 2: Performance Evaluation of Parallel Scheduler

Threads	**Execution Time (ms)**	**Speedup**	**Efficiency**
1	307.553	1.00	1.00
2	173.195	1.78	0.89
4	75.398	4.09	1.02
8	53.256	5.80	0.72
16	43.830	7.05	0.44

7.2 Performance Visualization

As illustrated in Figure 1, execution time decreases while speedup increases with the number of threads. The speedup grows rapidly at lower thread counts but gradually

stabilizes at higher levels due to overheads and limited parallelism in the DAG structure.

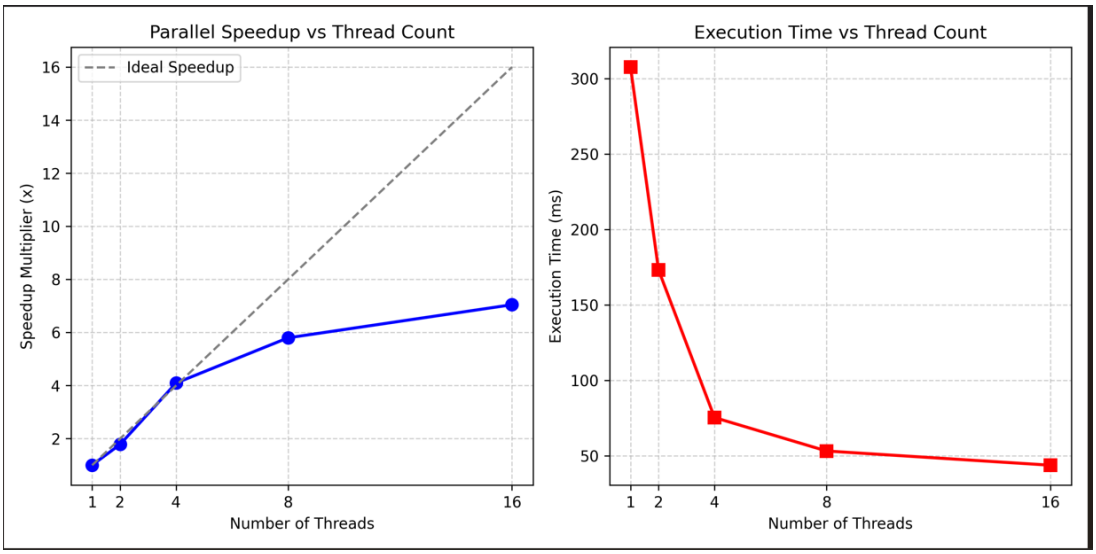

Figure 1: Execution Time and Speedup vs Thread Count

7.3 Discussion

As shown in Table 2 and Figure 1, the execution time decreases significantly as the number of threads increases. The system reduces execution time from 307.553 ms in the sequential case to 43.830 ms using 16 threads, demonstrating substantial performance improvement.

The rate of improvement from one to four threads is quite significant; it equals about 4.09x, which shows a proper utilization of available resources in terms of their parallelism. Moreover, the speedup further increases when there are eight and sixteen threads involved; its value is estimated at 5.80x and 7.05x, respectively.

In terms of the efficiency, the indicator achieves values close to unity when the number of threads is small. On the other hand, the efficiency decreases as the number of threads utilized by the algorithm grows. This result is not surprising since some overhead appears when working with threads and managing work stealing.

It may also be mentioned that the efficiency of the algorithm departs significantly from a linear function with growing numbers of threads. This situation can be explained by the lack of parallelism due to the DAG structure and the need for accessing shared resources.

In conclusion, it can be claimed that the proposed scheduling strategy works satisfactorily due to a quick runtime and good workload balancing through work stealing.

8 Conclusion

Our proposed solution uses Directed Acyclic Graph (DAG) method applied for task scheduling. The algorithm is parallel and performs task scheduling based on dependency and priorities. Using Directed Acyclic Graph (DAG) for task representation and applying multithreading in C++17 programming language allowed the successful implementation of the task graph.

The introduction of work-stealing techniques enabled the balanced distribution of the workload among threads and prevented idle time due to dynamic task distribution among processors. It was found out from the experiment performed that 7.05x speedup could be achieved with 16 threads.

Although the efficiency level of the system has decreased due to synchronization costs, yet, the efficiency level of the system remains highly efficient and scalable.

References

[1] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM (JACM)*, vol. 46, no. 5, pp. 720–748, 1999.

[2] A. Robison, "Intel threading building blocks," in *O'Reilly Radar Summit*, 2008.

[3] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Computing Surveys (CSUR)*, vol. 31, no. 4, pp. 406–471, 1999.

[4] N. S. Arora, R. D. Blumofe, and C. G. Plaxton, "Thread scheduling for multiprogrammed multiprocessors," *Theory of Computing Systems*, vol. 34, no. 2, pp. 115–144, 2001.